



Explore | Expand | Enrich

<Codemithra />TM

LOOP DETECTION



LINKED LIST

- A linked list is a collection of nodes that organize and store elements.
- Unlike arrays, which use a contiguous block of memory to store elements, linked lists use a series of nodes.
- This structure allows for efficient insertion and deletion of elements at any position within the list but may result in slower random access times.

LINKED LIST



LINKED LIST-STRUCTURE

1. Node: Each node in a linked list contains two fields:

Data: This field stores the actual data or values you want in the list.

Next (or Link): This field contains a reference (a pointer or link) to the next node in the sequence

2. Head: The head of the linked list refers to the first node in the list. It is the starting point for traversing the list.

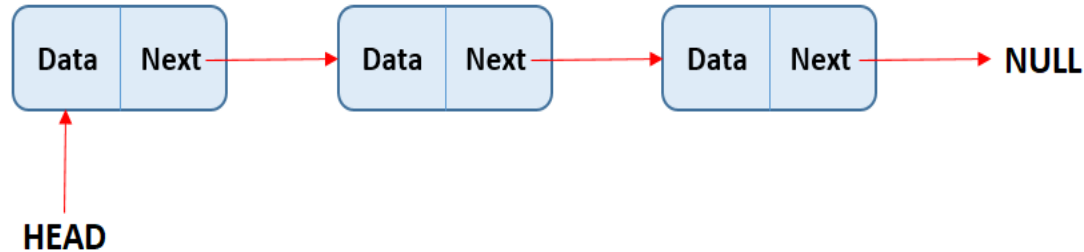
3. Tail: In some cases, linked lists may have a tail pointer, which refers to the last node in the list.

LINKED LIST

Types

1. Singly Linked List:

- Each node has a reference to the next node in the list.
- Traversal can only be done in one direction, from the head to the tail.

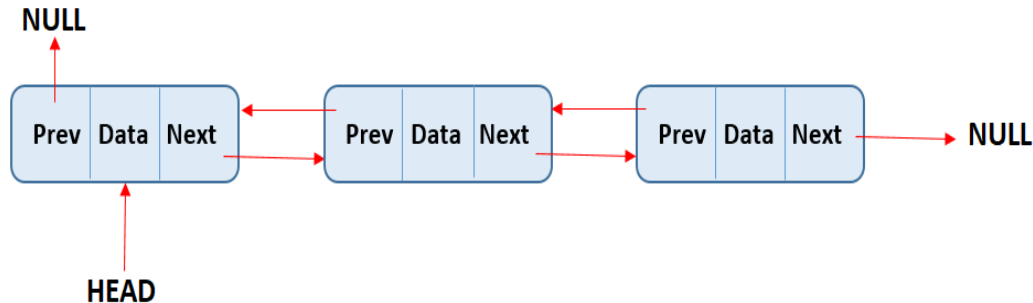


LINKED LIST

Types

2. Doubly Linked List:

Each node has references to both the next and the previous nodes in the list. This allows for bidirectional traversal but requires more memory

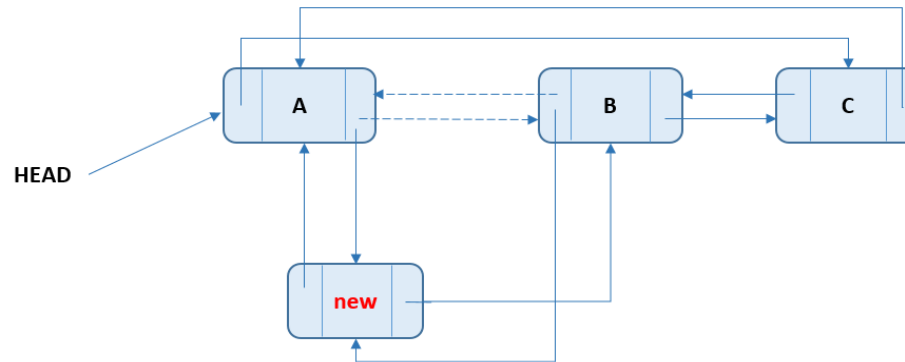


LINKED LIST

Types

3. Circular Linked List:

In this type, the tail node's reference points back to the head, forming a closed loop. It can be either singly or doubly linked.



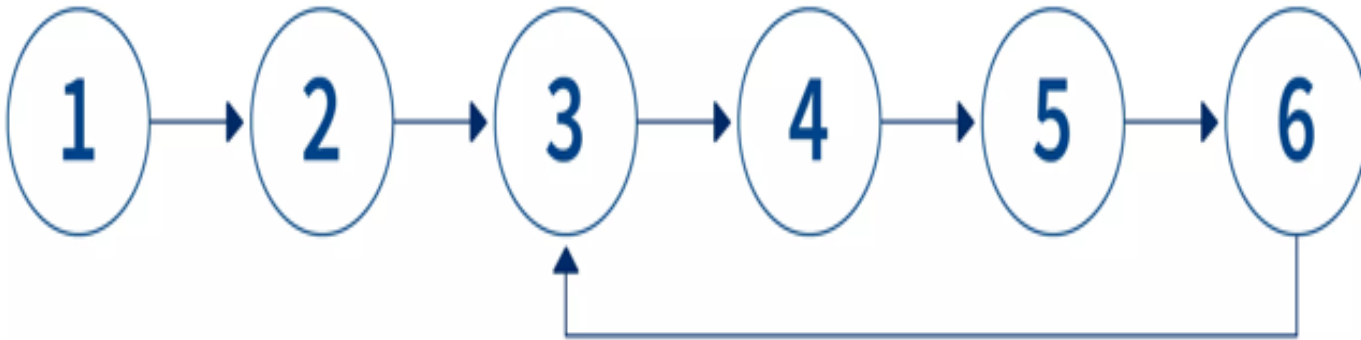
LOOP DETECTION

Loop & linked list cycle

- A circumstance that arises when a linked list has no ends is known as a loop in a linked list.
- When a loop is present, the last pointer in the linked list points to the linked list's head rather than the Null as is the case in a singly or doubly linked list and the circular linked list, respectively.
- The linked list cycle, commonly known as the loop, is present when it points to another node.

LOOP DETECTION

Loop & linked list cycle



LOOP DETECTION

Algorithm

1. Initialize two pointers, slow and fast, both pointing to the head of the linked list.
2. Traverse the linked list using the slow and fast pointers.
3. Move the slow pointer one step at a time and the fast pointer two steps at a time.
4. If there is no loop in the linked list, the fast pointer will eventually reach the end (become null) because it moves faster than the slow pointer. In this case, return "No Loop Detected."

LOOP DETECTION

Pseudocode

```
function detectLoop(head):  
    slow = head  
    fast = head  
  
    while fast is not null and fast.next is not null:  
        slow = slow.next           // Move one step at a time  
        fast = fast.next.next      // Move two steps at a time  
  
        if slow is equal to fast:  
            return "Loop Detected" // Cycle detected  
  
    return "No Loop Detected"      // No cycle detected
```

LOOP DETECTION

Algorithm

```
1 -> 2 -> 3 -> 4 -> 5
                        |
                        6
                        |
                    9 <- 8 <- 7
```



LOOP DETECTION

Explanation

1. Initially, both ``slow`` and ``fast`` pointers point to the head (node 1).
 2. Iteration 1: ``slow`` moves to node 2, ``fast`` moves to node 3.
 3. Iteration 2: ``slow`` moves to node 3, ``fast`` moves to node 5.
 4. Iteration 3: ``slow`` moves to node 4, ``fast`` moves to node 7.
 5. Iteration 4: ``slow`` moves to node 5, ``fast`` moves to node 9.
 6. Iteration 5: ``slow`` moves to node 6, ``fast`` moves to node 6.
 7. At this point, ``slow`` and ``fast`` are both at node 6, and they meet.
- The algorithm returns "Loop Detected."

LOOP DETECTION

Algorithm

5. If there is a loop in the linked list, the fast pointer will eventually catch up to the slow pointer within the loop. This is because it moves twice as fast and will "lap" the slow pointer within the loop.
6. When the fast pointer equals the slow pointer, return "Loop Detected."

LOOP DETECTION

Approach

Approach 1: Hashing

Approach 2: Floyd's Cycle Detection

LOOP DETECTION

Detect a loop in a linked list-hashing approach

Hashing in loop detection involves using a hash function to map unique identifiers (e.g., memory addresses) of visited nodes to distinct values in a data structure (e.g., a HashSet).

This allows efficient tracking and detection of previously visited nodes while traversing a linked list, helping identify loops by detecting nodes that appear more than once.

LOOP DETECTION

Detect a loop in a linked list-hashing approach

detect a loop in a linked list using a hashing approach.

Algorithm:

1. In this approach, we use a HashSet to store the references of nodes as we traverse the linked list.
2. If we encounter a node whose reference already exists in the HashSet, it means there is a loop in the linked list.
3. Otherwise, we continue traversing the list until we reach the end.

LOOP DETECTION

Detect a loop in a linked list-hashing approach

```
import java.util.*;

public class LinkedList {
    static Node head;
    static class Node {
        int data;
        Node next;
        Node(int d) {
            data = d;
            next = null;
        }
    }
    static public void push(int new_data) {
        Node new_node = new Node(new_data);
        new_node.next = head;
        head = new_node;
    }
}
```

```
static boolean detectLoop(Node h) {
    HashSet < Node > s = new HashSet < Node > ();
    while (h != null) {

        if (s.contains(h))
            return true;

        s.add(h);

        h = h.next;
    }

    return false;
}
```



LOOP DETECTION

Detect a loop in a linked list-hashing approach

```
public static void main(String[] args) {  
    LinkedList llist = new LinkedList();  
  
    llist.push(20);  
    llist.push(4);  
    llist.push(15);  
    llist.push(10);  
  
    llist.head.next.next.next.next = llist.head;  
  
    if (detectLoop(head))  
        System.out.println("Loop Exists");  
    else  
        System.out.println("Loop doesn't exists");  
}  
}
```

LOOP DETECTION

Detect a loop in a linked list-hashing approach

Time Complexity: $O(n)$ where n is the number of nodes in the linked list.

Space Complexity: $O(n)$ for the HashSet to store the visited nodes.

LOOP DETECTION

Detect a loop in a linked list-Floyd's Detection

detect a loop in a linked list using Floyd's Cycle Detection.

Algorithm:

1. In this approach, we use two pointers - one slow (tortoise) and one fast (hare).
2. The slow pointer moves one step at a time, while the fast pointer moves two steps at a time.
3. If there is a loop in the linked list, the fast pointer will eventually catch up to the slow pointer.
4. If there is no loop, the fast pointer will reach the end of the list.

LOOP DETECTION

Detect a loop in a linked list-Floyd's Detection

```
public class LinkedList {  
    //1.Node structure  
        static class Node {  
            int data;  
            Node next;  
            Node(int d) {  
                data = d;  
                next = null;  
            }  
        }  
    //2.push operation  
        Node head;  
        void push(int newData) {  
            Node newNode = new  
Node(newData);  
            newNode.next = head;  
            head = newNode;  
        }  
}
```

```
//3.detect loop  
void detectLoopAndPrint() {  
        Node slow = head, fast = head;  
        while (slow != null && fast != null &&  
fast.next != null) {  
            slow = slow.next;  
            fast = fast.next.next;  
            if (slow == fast) {  
                System.out.println("Loop  
found");  
                return;  
            }  
        }  
        System.out.println("Loop not found");  
    }
```

LOOP DETECTION

Detect a loop in a linked list-Floyd's Detection

```
//4.main method
public static void main(String[] args) {
    LinkedList list = new LinkedList();

    list.push(20);
    list.push(41);
    list.push(5);
    list.push(10);
    //5.Creating a loop for testing
    list.head.next.next.next.next = list.head;
    list.detectLoopAndPrint();
}
}
```



LOOP DETECTION

Detect a loop in a linked list-Floyd's Detection-Time & space complexity

Time Complexity: $O(n)$ - Linear time complexity, where n is the number of nodes in the linked list.

Space Complexity: $O(1)$ - Constant space complexity, as it uses a fixed amount of additional memory regardless of the size of the linked list.

LOOP DETECTION

Hashmap vs Floyd's Detection

Detecting a loop in a linked list using a HashSet has a time and space complexity of $O(n)$ due to the need to store all visited nodes, while Floyd's Cycle Detection has a time complexity of $O(n)$ and a space complexity of $O(1)$ as it uses two pointers, making it more memory-efficient for large lists. Floyd's algorithm is generally preferred for loop detection in linked lists.

INTERVIEW QUESTIONS

1. How do you detect a Loop in a Linked List?

You can detect a loop in a linked list using techniques like Floyd's Cycle Detection (Tortoise and Hare) algorithm or using a HashSet to track visited nodes. The former has a time complexity of $O(n)$ and space complexity of $O(1)$, while the latter has a time complexity of $O(n)$ and a space complexity of $O(n)$.

INTERVIEW QUESTIONS

2. Explain the Floyd's Cycle Detection algorithm.

Floyd's Cycle Detection, also known as the Tortoise and Hare algorithm, uses two pointers - one slow and one fast. The slow pointer moves one step at a time, while the fast pointer moves two steps at a time. If there's a loop, the fast pointer will eventually catch up to the slow pointer. If there's no loop, the fast pointer will reach the end of the list.

INTERVIEW QUESTIONS

3. Can you implement Loop detection using a HashSet?

Yes, you can implement loop detection using a HashSet. While traversing the linked list, you store references to visited nodes in a HashSet. If you encounter a node that's already in the HashSet, you've detected a loop. Otherwise, you continue traversing. This approach has a space complexity of $O(n)$.

INTERVIEW QUESTIONS

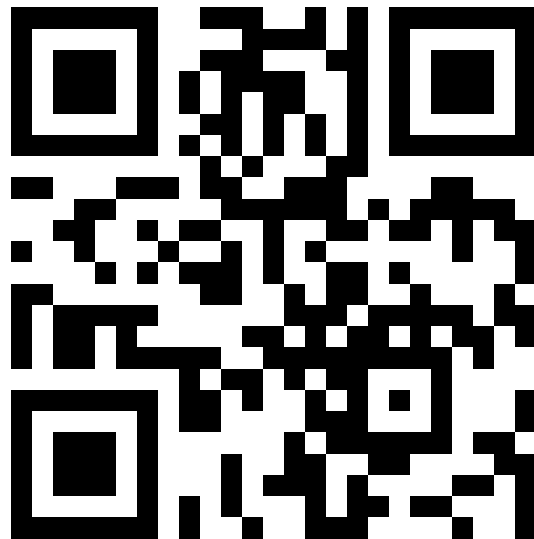
4. What is the time and space complexity of loop detection using Floyd's Cycle Detection?

Floyd's Cycle Detection has a time complexity of $O(n)$ because it makes at most two full passes over the list. Its space complexity is $O(1)$ because it only uses two pointers, regardless of the list's size.

INTERVIEW QUESTIONS

5. Explain how you would remove a loop in a linked list.

To remove a loop in a linked list, you can use Floyd's Cycle Detection to find the meeting point of the two pointers. Then, reset one pointer to the head and move both pointers one step at a time until they meet again. The point where they meet is the starting node of the loop. Set the `next` of the previous node to null to break the loop.



<https://learn.codemithra.com>



THANK YOU